

Recruitment Scam Shield

A focused, agentic AWS hackathon plan for preventing fake job and impersonation scams in India

Combined project brief • research • shipping plan • architecture • problem directions

Prepared for the AWS First Commit open build challenge

Contents

1. Selected problem and product thesis
2. Background research and scam patterns
3. First working loop and delivery timeline
4. AWS architecture and service choices
5. Demo, repository, risks, and team split
6. Earlier problem shortlist (appendix)

Recruitment scam prevention — three-person build report

Updated delivery plan: read First-loop shipping plan for the current timeline, architecture and service decisions. This document remains the background research report; its earlier sequencing is superseded.

Research and proposed scope: 16 September 2026. This is a planning document, not a claim that the application or integrations have been built.

1. The decision

Build an AWS-hosted website that helps someone check a recruitment or placement offer **before paying, handing over sensitive documents, or joining**.

The user submits the offer, recruiter details and any links. Our agent checks independently sourced employer information, investigates the claims, asks for missing evidence, and gives a supported recommendation. The user can then prepare a report or continue through the employer's official channel.

The promise: **"Check who is offering the job, what they can actually promise, and what you are being asked to give up."**

Support two forms of the same problem:

- Someone impersonates an employer or recruiter to sell a fake job.
- An intermediary sells training, unpaid work or membership using a claimed route into a named employer that it cannot substantiate.

Paid-task offers are covered at entry: an apparent job requires funding tasks, buying access or recruiting others. We are not building a tool for managing MLM participation. Paid-task fraud and MLM are different categories and should not be labelled interchangeably.

The main outcome is a better decision before commitment. Later evidence can update an existing case, but users should not need to have already lost money for the product to help.

2. Why this is a real problem

A March 2026 parliamentary response lists 11,126 reports under "Online Job Fraud" in 2025, compared with 10,457 in 2024. This is a particular NCRP reporting category, not a count of every recruitment or task scam. MHA response, page 4

I4C's April 2025 advisory describes fake CAPTCHA jobs promoted through social media, job portals, messages and email. Victims encounter fabricated agreements, fees, denied earnings and further payment demands. I4C advisory

Amazon India separately documents impersonators seeking payments for laptops, uniforms, training and other supposed recruitment requirements. Its published policy says it does not charge recruitment fees. This makes Amazon a useful first supported employer, rather than the entire product. Amazon recruitment warning

These sources establish a continuing problem. They do not establish that recruitment is India's biggest scam or that our product is better than existing tools. We should test with a few job seekers and a placement coordinator, using redacted examples, before finalising the interface.

3. What we can conclude about an offer

Use three outcomes:

- High-risk evidence found: state the specific contradiction or dangerous request and recommend pausing.
- Important claims remain unverified: identify what evidence is missing and how to check it independently.
- Checks completed; no contradiction found: explain the limited checks performed. This is not a guarantee that the offer is safe.

Missing proof of guaranteed placement does not automatically prove fraud. A training provider may legitimately offer interview preparation, referrals or placement assistance. Distinguish those services from a claim of guaranteed employment.

For a claim such as “Work with us for two months and get an Amazon job,” extract:

Claim	Evidence to seek
Official hiring partner	Confirmation from the named employer, an employer-published partner listing, or independent confirmation through an official contact
Guaranteed job	Written terms: who employs the candidate, conditions, selection requirements and what happens if placement fails
Guaranteed interview	A defined interview arrangement; distinguish it from guaranteed selection
Placement success rate	Time period, cohort size, number placed, definition of placement and independently checkable support
Refund promise	Written eligibility, exclusions, deadline and actual contracting entity
Work before placement	Written explanation of duties, pay, supervision and relationship to the supposed employer

Testimonials and logos are claims, not proof of a partnership. Even evidence of past placements cannot guarantee a future job. Do not request other candidates' private records or scrape personal profiles to manufacture certainty.

4. The full user journey

- Open the website and choose Check an offer . A labelled sample is available without sign-up.
- Paste the message or upload a screenshot. Optionally add an offer PDF, recruiter email and job/placement URL.
- Review the extracted claims and correct mistakes before investigation.
- The agent verifies the employer, vacancy, claimed affiliation and requested payment or documents.
- If an important fact is missing, it asks one focused question, such as “Can you share the written placement guarantee?”
- The results show the recommendation, supporting evidence, unresolved claims and official next step.
- Choose Report to our team , Prepare an authority report , or Open the official application route .
- Add later evidence to the same case if necessary.

Do not require Aadhaar, PAN, bank statements or an actual payment to use the checker. Allow users to redact uploads and delete their case.

5. How we verify recruiters and job portals

There is no universal public API that proves a person is an employer's HR representative. Our workflow builds a chain of evidence and stops where the evidence stops.

Check	Implementation	Limit
Employer identity	Start from a curated official domain and careers link	A real company can be impersonated
Vacancy	Compare role, job ID, location and application URL with the official listing	A real vacancy can be copied; an absent listing may be closed or private
Recruitment platform	Follow the link from the employer's own site to its applicant-tracking system	A platform's name alone does not establish employer ownership
Recruiter contact	Compare actual sender/reply-to domains and published employer guidance	A familiar display name proves nothing; legitimate agency recruiters also exist

Check	Implementation	Limit
Hiring process	Retrieve dated employer policies on fees, documents and candidate portals	A different interview sequence is weak evidence; processes vary by role
Claimed partnership	Look for employer-side confirmation	An intermediary's own partner page is not independent confirmation
Payment demand	Compare what is requested with employer policy and the promised service	Paid training or an official examination fee is not automatically fraud
Prior reports	Retrieve relevant advisories and reviewed reports	An allegation is not a conviction; no report does not mean safe

For email files, parse useful metadata, but do not trust arbitrary uploaded authentication headers as independently verified. A screenshot cannot prove email authentication. Domain authentication, when reliably obtained, establishes domain control rather than the truth of an offer.

Start with Amazon and one or two additional employers selected after confirming accessible official sources. Maintain employer-specific records; do not generalise Amazon's rules to every employer.

6. APIs, scraping and social reports

Prefer structured public APIs where they exist, then permitted public-page retrieval, then user-supplied evidence.

Source	How we use it in three days
Official employer careers/policy pages	Fetch a small allowlist, cache it, retain URL, timestamp and content version
Greenhouse Job Board API	Read published vacancies for a supported employer board linked from the official employer site; GET endpoints need no authentication
Lever Postings API	Read a supported employer's published jobs; it is not a global job-search API
LinkedIn, Indeed, Naukri and similar portals	Accept user-supplied listing links/text; add direct integrations only if documented access is available. Do not depend on universal free search APIs
I4C, CERT-In, employer fraud warnings	Scheduled retrieval of selected advisories and policies
FTC and other overseas authorities	Learn scam mechanics with country and date labels; never import foreign reporting instructions as Indian guidance
Reddit/social posts	Optional leads and links, accessed only through permitted methods. Store allegations separately and corroborate them
Reports submitted on our website	Private evidence queue with review status, deduplication and corrections

Greenhouse and Lever provide useful published-vacancy interfaces. LinkedIn's Job Posting API is restricted to approved developers and is for posting jobs, not freely searching every vacancy. Greenhouse documentation, Lever documentation, LinkedIn documentation

Cross-posting can help compare job IDs and details, but three copies of one advert are still one underlying claim. Do not increase confidence simply because the same text appears on several websites.

For scraping, respect site access rules, rate limits and terms; cache results and fetch only relevant pages. Do not bypass login, CAPTCHA or access blocks. If a page fails, use a dated cached source or ask for the document and label the limitation. Reddit access has its own conditions; it is not a guaranteed free data feed. Reddit API terms

7. One agent, two connected loops

Investigation loop

Extract claims → identify an unanswered question → choose a verification tool → inspect evidence → update findings → ask for evidence or finish.

Example: finding a real vacancy leaves recruiter affiliation unresolved. The agent then checks the claimed partnership or hiring policy. If a payment demand conflicts with that policy, it returns the exact contradiction and source.

Use a bounded loop: initially no more than six tool calls and one focused follow-up question per run. Each finding must reference an evidence ID. Ordinary code validates the output schema and evidence references before displaying it. If tools fail, return an incomplete result rather than manufacture an answer.

Knowledge-update loop

New advisory/report → store and deduplicate → extract a candidate pattern → corroborate → evaluate → approve/version → use in later cases.

DynamoDB stores new evidence. DynamoDB Streams can trigger Lambda processing; the database does not train a model by itself. Process stream events idempotently so retries do not create duplicate patterns or notifications. AWS event-source mappings

Official source changes become candidate updates. The agent compares old and new text and records the scope and effective date when known. A changed fee policy should not silently create accusations against earlier offers. Review substantive changes before activation.

User reports remain unverified until corroborated and reviewed. Multiple copied posts do not count as independent evidence. Keep a correction route and no public blacklist in the MVP.

The system improves its retrieved knowledge and detection patterns; it does not automatically retrain model weights. Evaluate proposed patterns against unseen examples and benign cases before publication. Record version, source, measured results and rollback target. A human reviewer approves new accusation-producing rules.

For the demo, import one new authoritative advisory, show the proposed update and evaluation result, approve it, then rerun a related case. Show actual outcomes, including if the update adds evidence without changing the verdict. Never stage an artificial failure just to claim improvement.

8. Channels and voice

Channel	MVP access
Website	Pasted text, screenshots, a job URL and bounded document uploads
Gmail	Paste or upload an email; optional extension captures user-selected visible content
WhatsApp/Telegram	Paste or screenshot evidence. A future bot receives messages explicitly sent to it; it cannot read unrelated private chats
Browser extension	Stretch goal: select offer text and current URL, preview the capture and send it to the same backend
Phone calls	No live-call monitoring. Users can describe a call in text

Broad Gmail API reading can require restricted-scope verification and a security assessment depending on data handling. That is unnecessary for this MVP. Google scope documentation

An AWS-backed WhatsApp bot is possible, but introduces onboarding and messaging fees. Keep it out of the critical path. AWS WhatsApp integration, pricing

Polly is useful for text-to-speech: a button can read the already generated explanation aloud. It is much simpler than conversational voice. It still adds voice/language selection, synthesis latency and playback work. Add it only after the deployed flow passes. Text must always remain available; a synthesis failure must not block the result. Polly does not listen to or join calls. Polly pricing

9. The website and intelligent reporting

The result page should answer four questions: **What should I do? What did you verify? What remains unknown? How can I check or report this?**

Use a clear recommendation followed by cards for identity, vacancy, affiliation, payment and placement claims. Each card shows the evidence, source and retrieval date. Show an activity log of checks performed, not private model reasoning or invented confidence percentages.

Example: “The vacancy is real. We could not verify this agency’s partnership. Its guaranteed-placement claim has no employer-side evidence. Ask for written terms and verify through the employer before paying.”

Reporting should reuse extracted details: website, sender, claimed employer, requested amount, timeline and evidence. The user reviews the description and redactions before submission to our private database. Issue a case ID and show review status.

For authorities, generate a concise summary and evidence checklist and open the official reporting route. I4C accepts reports of suspicious identifiers, while cybercrime victims can report through NCRP/1930. OTP/CAPTCHA steps mean we should not promise unattended submission. “Prepared” and “submitted” must be separate statuses; only display the latter with a real acknowledgement. I4C Report Suspect, complaint requirements

Add three case-specific prevention tips, such as checking the actual application destination, independently confirming a claimed partnership and distinguishing placement assistance from guaranteed employment. Keep advice short and tied to the evidence. An optional private, expiring summary can help someone consult family without exposing all uploads.

10. AWS stack and why each piece exists

Use a React/TypeScript frontend and a Python backend. Avoid adding services just to fill a diagram.

Service/tool	Purpose
Amplify Hosting	HTTPS website and a stable public URL
API Gateway	Browser-to-backend API with request limits
Lambda	Intake, source retrieval, bounded agent jobs and reporting functions
Bedrock	Model for extracting claims, selecting checks and explaining evidence; choose an accessible model after a small latency/cost evaluation
Strands Agents SDK	Open-source agent loop with a small set of explicit verification tools
DynamoDB	Cases, job status, employer records, reports and pattern versions
DynamoDB Streams	Start processing when a relevant new record arrives; filter events to avoid update loops
S3	Private uploads, source snapshots and report exports; signed access links and retention rules
EventBridge Scheduler	Refresh a small official-source allowlist on a schedule
Cognito	User accounts for real saved cases and reviewer access; anonymous demo sees only synthetic cases
CloudWatch	Errors, latency and usage logs without raw personal evidence
AWS SAM CLI	Versioned infrastructure and reproducible deployment; local backend development
Polly, optional	Read the result aloud

Keep the investigation asynchronous: create a job, let the worker update status, and have the browser poll. Do not depend on holding one long API request open.

Step Functions is optional if durable multi-stage ingestion needs it; a small persisted job workflow is enough initially. OpenSearch is unnecessary for a small curated library. Cedar, Firecracker, Corretto, PartyRock and LocalStack are not needed to complete this workflow. Strands and SAM already provide purposeful use of the open-source stack.

11. Cost and reliability: promises we can make

The goal is no paid third-party dependency and no out-of-pocket cloud usage while confirmed event credits cover the demo. AWS inference, storage and hosting are metered. Zero cost forever, zero payment-method requirements and perfectly deterministic AI conclusions cannot be guaranteed.

The event page advertises team credits and notes that a new AWS account can have a small verification charge. Confirm credits, expiry, eligible services and model access immediately. Keep the deployed app available through the judging period, not only while recording. Event page

Use provider URLs, public job APIs and permitted source retrieval instead of paid search/scraping services. Cap uploads, input size, model output, tool calls and daily investigations. Cache public evidence, limit worker concurrency and provide an application kill switch. Budget alerts are warnings, not an instant hard spending cap. Account separately for hosting, logs and storage even when investigations are disabled.

Make the workflow predictable: validated inputs, exact domain parsing, bounded retries, source timestamps and defined error states. AI interpretation remains probabilistic. A blocked source must yield “could not check,” not “fraud.”

Keep a labelled reproducible sample mode using synthetic offers and dated public-source snapshots. The agent actually runs on those inputs. Mark each evidence item as live, cached or sample. Never present cached checks as live integrations.

Treat uploaded messages and retrieved pages as untrusted data. Do not execute their instructions, visit private-network addresses, submit credentials or download executables. Prefer allowlisted official verification sources; arbitrary suspicious URLs can be parsed without fetching. Keep credentials server-side and isolate each user’s evidence.

12. Why investment scams remain a valid alternative

We can build for investment scams. Their common entry points include social ads, messaging groups, impersonated advisers and fake trading sites/apps. A web product can examine pasted group messages, adviser claims, broker domains and browser-visible requests. It cannot automatically observe private groups, native trading apps or bank transfers.

A narrow alternative would verify a claimed adviser/broker relationship before the first deposit. It would use SEBI/exchange records and official broker-domain evidence. A registered entity does not authenticate the person, group or platform using its name. NSE investor advisories

Investment fraud has the stronger reported financial-harm case. Recruitment is the chosen MVP because its employer-process checks and pre-payment outcome are easier to explain and validate with our peers in three days. Supporting both now would double the domain-specific verification work. Keep investment fraud in the future-work section only.

13. Three-person plan

Agree first on shared request/result schemas, evidence IDs, verdict definitions and sample cases. Person C owns integration and releases, but all three review changes and test the full flow.

Owner	Day 1	Day 2	Day 3
A — product/frontend	Intake, result layout, sample cases and API client	Evidence cards, follow-up question and reporting form	Accessibility, loading/error states, demo recording
B — agent/evidence	Extraction schema, bounded tools, first employer policy adapter and one public job API adapter	Impersonation/placement-claim checks, evaluation cases and update proposal	Fix false positives, verify sources, document model limits
C — AWS/integration	Deploy frontend and one backend vertical slice; configure auth, tables, storage and CI	Async jobs, private reports, scheduled refresh and reviewer update flow	Cost controls, security/reliability checks, deployment documentation and release

End of day 1: the public URL accepts a sample and returns a sourced result from one real backend check. End of day 2: intake → investigate → explain → report works, with one controlled knowledge update. Day 3: evaluate, fix, record and submit. Freeze optional features early.

Cut order if time slips: Polly → extension → extra employers → advanced clustering. Preserve the deployed investigation, evidence-backed result and reporting path. A two-person peer review of each result is more valuable than another unsupported integration.

14. Roadblocks and planned responses

Roadblock	Response
Bedrock model/region unavailable	Test first; use another accessible model after a small quality check. Escalate credit/account access to organisers early
Career site blocks access	Use supported public API or dated snapshot and clearly show the limitation
Recruiter identity cannot be proved	Return unverified affiliation and an independent verification route
Training company makes vague guarantees	Show the exact promise and missing evidence; avoid an unsupported fraud accusation
Employer process changes	Version the source, flag the change and review before activating new policy rules
Malicious or duplicate reports	Private queue, deduplication, corroboration and reviewer approval
New pattern increases false positives	Reject or roll back the pattern version
API/model timeout	Persist state, retry within limits, show partial evidence and allow retry
People will not paste evidence	Test intake with peers; improve paste/upload flow before adding integrations
Extension/store review is slow	Keep website complete; show optional unpacked extension only in the recording

15. Public repository and PR workflow

There is no verified special repository style that wins this event. Make the implementation easy to inspect and reproduce. Do not manufacture commits, empty PRs or fake review activity for appearances.

Use one repository with clear folders: web/, backend/, infra/, evals/, docs/ and optional extension/.

Open small issues with acceptance criteria, then short-lived feature branches such as feat/offer-intake, feat/employer-checks and feat/report-pipeline. Open draft PRs early. Link the issue and include the problem solved, behaviour, relevant screenshots, validation and limitations.

One teammate reviews each PR. Run focused CI checks: type checks/build, unit tests for important parsers and decision rules, and evaluation regressions for changed patterns. Use synthetic data. Merge small changes into main daily; squash messy working commits if useful and preserve correct authorship. Require a passing check and reviewer approval where the repository plan supports it.

Use GitHub Actions OIDC with a restricted AWS role instead of storing long-lived AWS keys. Ensure untrusted fork PRs cannot obtain deployment permissions. Deploy from the protected main branch. Tag the submitted commit and make the app's build version visible to reviewers.

The README should start with the problem, deployed URL, video and a one-minute sample walkthrough. Include setup commands, environment-variable names, infrastructure deployment, architecture, source provenance, privacy/retention behaviour, limitations, actual evaluation results and observed costs. Add an .env.example, suitable licence, dependency lockfiles and each person's contribution/learning notes. Never commit real victim data, credentials or unredacted reports.

16. What the judges should be able to verify

The supplied rubric rewards real impact, working execution, AWS use, learning and a three-minute recorded demo. It does not reward service count or elaborate git history. The event page also says project work starts when the event clock starts; keep this preparatory report separate and confirm any uncertainty about preparation with the organisers. Official rubric and FAQ

Evaluate a small held-out set containing impersonation, unsupported placement guarantees, task-entry offers, legitimate recruitment, legitimate paid training, expired vacancies and unavailable sources. Record false accusations, missed risky requests, unsupported claims, citations, latency and cost. Test failure handling, not just successful demos.

Have several peers try unfamiliar cases. Observe whether they understand what remains unknown and can choose a safe next action. Do not claim money saved unless it was actually measured.

Three-minute video:

- 0:00–0:20: Show a fictional job seeker facing a persuasive offer before payment.
- 0:20–1:10: Submit the offer; watch the agent verify a real vacancy but leave affiliation unresolved, then find a concrete contradiction.
- 1:10–1:35: Show the evidence and official verification route; prepare a report.
- 1:35–2:00: Show a placement-guarantee claim with missing proof and a benign case that is not falsely accused.
- 2:00–2:30: Demonstrate one new advisory moving through evaluation and approval into a versioned knowledge update.
- 2:30–3:00: Show the AWS flow, actual evaluation/cost results and one learning from each teammate.

Use synthetic cases clearly marked as samples. The public app must expose the same core capability seen in the video, with fresh sessions that cannot see other users' evidence. Test its URL in an incognito browser. Confirm submission links, repo visibility, video permissions and deployment lifetime before submitting.

The finished feature is one complete loop: **offer submitted** → **claims investigated** → **evidence explained** → **candidate can verify independently or prepare a report before committing**. The knowledge-update path supports that same feature. Voice, broad social crawling, live phone monitoring and investment investigations do not belong in the three-day critical path.

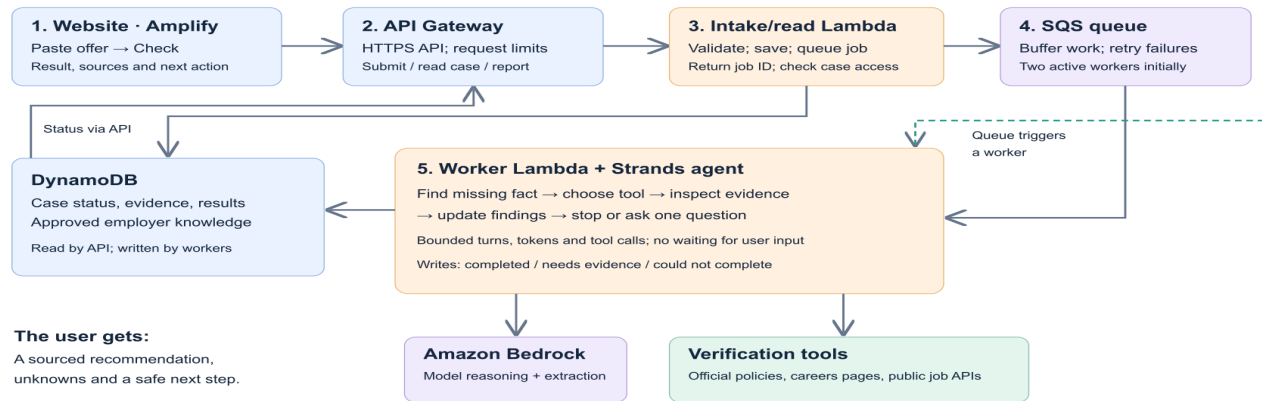
Architecture diagram

The diagram below shows the synchronous user check, asynchronous knowledge refresh, and the AWS services that keep the loop auditable and scalable.

Check a recruitment offer before committing

Proposed AWS architecture • solid arrows: first deployed loop • dashed arrows: later knowledge refresh

FIRST WORKING LOOP — SHIP ON DAY 1



AFTER THE CORE WORKS — UPDATE WHAT THE AGENT KNOWS



User reports enter a private review queue. They never become confirmed fraud labels automatically.

Across the system: IAM limits permissions • CloudWatch shows failures and latency • SAM defines deployment

Add S3 only for uploads/snapshots. Cognito only for saved accounts. Polly and the extension are optional.

Recruitment scam checker: build the first useful loop, then improve it

Revised 16 September 2026. This is the active delivery plan; it replaces the sequencing and service choices in the earlier research report. No application has been implemented by writing this plan.

Guiding source: Aayush Sharma's From First Idea to Working Demo, supplied by the user. We follow its sequence: define the user and output, reduce to one flow, pick only necessary services, deploy end to end first, and freeze on the fourth day.

1. Four answers before code

Who? An Indian student or first-time job seeker assessing an offer before paying or sharing identity documents. Our design example is a fictional final-year student offered a role through someone claiming to represent a large employer. Validate this example with a real peer; do not present it as an interviewed victim.

When? A recruiter or placement company requests a payment, documents, work or course enrolment while promising employment with a named company.

Today? The candidate searches the company name, asks friends and checks whether a similar vacancy exists. Those checks may validate a real company or copied job advert while leaving the sender's relationship unverified. Treat this as a user-research hypothesis to test with peers.

What comes back? One page containing a practical recommendation, cited evidence, unresolved claims and a safe verification/reporting step.

The product promise: **"Check the offer before you commit."**

2. The one flow that must work

Paste an offer and optional recruiter/job URL → investigate its claims against an official employer source → return a sourced recommendation → help the candidate verify independently or prepare a report.

The first case is employer impersonation with a recruitment payment demand. Amazon is a suitable first employer because it publishes a no-fee recruitment policy. A matching real vacancy must not authenticate the sender. Amazon recruitment warning

The next case extends the same flow to an intermediary promising placement in a named company after training or work. Missing proof means **unverified placement claim**, not automatically "scam." Legitimate paid training, placement assistance and guaranteed employment must remain distinct.

Paid-task schemes are checked at entry if visible terms require the candidate to fund work. Do not require prior participation or a loss to use the app. Do not build investment checking, MLM management or call monitoring in this submission.

3. Architecture the team can read at a glance

The solid-arrow path is the day-one system. The green path is added only after that system works. The diagram is a proposed design, not evidence of an existing deployment.

What happens after someone clicks Check

- Amplify serves the website. The candidate submits text to API Gateway.
- The intake Lambda validates length, permitted inputs and access limits. It creates a case in DynamoDB and sends its job ID to SQS.
- The API returns a job ID quickly. The page shows Queued , then polls for status. It does not keep a long request open while the model works.
- SQS triggers a worker Lambda. The worker loads the case and a pinned version of approved employer knowledge.

- Strands runs the bounded investigation, using Bedrock and explicit verification tools. It saves progress and evidence in DynamoDB.
- The worker writes Completed , Needs evidence , or Could not complete . The API returns that saved state to the browser.
- The candidate can create a report draft from those findings. External reporting remains user-reviewed and uses the official route.

SQS adds a real dependency, so implement it from one SAM template. Its purpose is concrete: the website stays responsive while slow checks wait, and the team can limit simultaneous model work.

4. What every service does, and when to add it

Service or tool	Why we need it	When
Amplify Hosting	Serves the React website through HTTPS and gives judges a public URL	First deployed loop
API Gateway	Receives submissions and status/report requests; applies basic request throttles	First loop
Lambda	Small intake/read functions plus a worker that actually investigates the offer; runs on demand	First loop
SQS	Holds job IDs until workers are available; retries transient failures and isolates failed messages	First loop
DynamoDB, on-demand	Stores case state, evidence references, results, employer policies and pattern versions	First loop
Bedrock	Model access for claim extraction, deciding the next check and explaining evidence	First loop, after confirming account/model access
Strands Agents SDK	Open-source library inside the worker; carries out the model/tool/result loop	First loop; it is not another hosted server
IAM	Gives each function only the AWS actions/data it needs; protects deployment roles	From the start
CloudWatch	Records errors and latency; alarms on a growing queue or failed jobs	From the start; avoid logging personal offer text
SAM CLI / template	Defines API, functions, queue and database in the repo so teammates can reproduce deployment	From the start
S3	Private screenshots/documents and dated source snapshots; browser uses short-lived upload links	Day two, only when adding files or snapshots
EventBridge Scheduler	Starts a daily refresh of selected official sources	Day three, after manual knowledge import works
Cognito	Accounts for persistent private cases and roles	Optional; no login screen needed for synthetic sample demo
Polly	Speaks the result the user already sees	Last optional addition; no call joining or listening

Step Functions would help coordinate a larger durable pipeline, but is unnecessary for this bounded three-day workflow. DynamoDB Streams can later trigger automatic report processing; we will initially save reports and run an explicit reviewer import instead. This avoids two overlapping ways to start the same work. No OpenSearch cluster, App Runner server, Cedar policy layer, Firecracker setup, PartyRock prototype or LocalStack requirement is needed for the chosen flow.

5. The two loops, precisely

Loop A: investigate one offer

Starts: on Check, or when the user adds requested evidence and clicks Check again. Polling for status never reruns the model.

Repeats: extract a claim → determine the missing evidence → select a tool → read the result → update the finding. Tools are limited to approved-source retrieval, published vacancy lookup, domain comparison and evidence retrieval. Uploaded content cannot grant the agent tools or permissions.

Stops: when it has a supported recommendation, needs the user, hits its configured limits, or a required check fails. Start with at most four model turns and six tool calls, an input/output token allowance and a worker deadline. Tune these with actual cases.

If the user must supply evidence, save the state and end the invocation. The next submission creates a new case revision. Do not keep a Lambda running while waiting for a person.

Pin the source/pattern version for each run so a background update cannot silently change an in-progress investigation. Save only concise findings and tool evidence for users, not private model reasoning.

Loop B: improve the evidence used by future investigations

First implementation: a team member imports one authoritative advisory through a local authenticated command. No admin dashboard is required.

Later trigger: EventBridge starts a daily fetch of a small source allowlist. User reports go to a separate private review queue. Neither activity runs every time a visitor opens the website.

Pipeline: fetch → compare content hash → skip unchanged pages → extract a proposed update → evaluate on held-out scam and benign cases → review substantive changes → publish a version.

Stops: when the source is unchanged, inaccessible, the evaluation fails, reviewer action is required, or the version is published. No worker remains active waiting for approval. The reviewer's later command resumes the publication step.

New investigations use the approved version. Existing results retain their version and offer an explicit recheck. No automatic mass re-analysis of every saved case.

This updates retrieval and patterns, not model weights. Reports are allegations until corroborated; attackers must not be able to train the system by repeatedly reporting a legitimate recruiter. Demonstrate a real update, and report whether it improves accuracy, evidence quality or neither.

6. Sources and verification scope

Begin with one employer policy adapter and one official vacancy source. Add a second employer only after the first complete case works.

- Establish the official employer domain independently of the offer. Follow its careers link to an applicant-tracking platform.
- Match job ID, role and location where available. Missing/closed/private listings remain uncertain.
- Verify a claimed recruiter/agency relationship separately. Employer-side partner confirmation is stronger than logos or self-published testimonials.
- For placement promises, extract written conditions, fees, refund exclusions, who will employ the candidate and whether the promise is assistance, interview access or employment.
- Check employer policy dates and scope. An unusual interview sequence alone is weak evidence of fraud.

Use public APIs before scraping: Greenhouse GET endpoints expose published job-board data without authentication; Lever exposes published employer postings. Verify the board's relationship to the employer. Neither is a universal recruiter-identity service.

Cross-posted listings are supporting context, not independent proof. LinkedIn's Job Posting API requires approval and is not an unrestricted vacancy-search API. Do not make LinkedIn, Indeed or Naukri API approval a delivery

dependency.

Retrieve a small set of permitted public pages with timeouts, size caps and caching. A blocked page yields “Could not verify” or an explicitly dated cached result. No CAPTCHA bypass, login scraping or paid proxy/search service.

Reddit and social posts can suggest new tactics, but require permitted access and corroboration. Keep broad crawling out of the MVP. Overseas government advisories can inform mechanics; keep country labels and use Indian reporting guidance for Indian users.

7. First-day schedule: reach the public working loop

The event runs 17–20 September. Use Thursday–Saturday for building; Sunday is the freeze, recording and submission day. The hour targets below are planning targets, not promises of latency or completion.

Time from start	Result the whole team must be able to demonstrate
First 45 minutes	One fictional offer, one official reference, one result schema and three acceptance cases agreed
By hour 2	Amplify URL calls a deployed API; a synthetic case is stored and read back. A labelled placeholder result is acceptable at this integration milestone
By hour 4	SQS runs the worker; the worker retrieves an actual employer policy and calls Bedrock. Remove placeholder verdicts
By hour 6	End-to-end: paste offer → queued → check → sourced result. Source failure gives a readable incomplete result
End of day	A teammate opens the URL in a fresh browser, runs a new input and can inspect why the result was produced

If this loop is not working by day’s end, all three work on it. Do not compensate by building more screens or adding another integration.

8. Additions ranked by value

Priority	Addition	Why it improves the same outcome	Acceptance gate
1	Evidence cards, clear uncertainty and official next action	Helps the candidate decide rather than merely receive a label	Every finding has evidence or is explicitly unresolved
2	Report draft + private report submission	Closes the loop when the user finds a suspicious offer	Reviewable draft, case ID and honest submission status
3	Unsupported placement-claim check	Covers intermediaries without starting another product	Paid training is not falsely labelled a scam solely for charging money
4	One follow-up question and evidence revision	Resolves missing facts without guessing	Saving/reopening the case preserves evidence and versions
5	Screenshot input through S3	Makes real offers easier to submit	Extraction can be corrected; invalid files are rejected
6	Controlled knowledge import, evaluation and approval	Shows adaptation with measurable evidence	A new version is tested, approved and used by a later case
7	Scheduled source refresh	Automates a pipeline already proven manually	Unchanged content causes no model call
8	Extension or Polly playback	Reduces capture/reading effort	Core website remains complete without either

A clever refresh pipeline cannot rescue a broken candidate workflow. Finish the candidate result and reporting path first.

9. What users and judges see

One main page: offer input on the left, investigation result on the right. Show real states such as Queued, Checking employer policy, Comparing the request and Complete. Do not invent progress percentages.

The result contains recommendation, verified facts, contradictions, unresolved claims, dated source links and next action. For example: “The vacancy exists. This sender’s affiliation remains unverified. The requested recruitment payment conflicts with the employer’s published policy.”

For a training-to-placement offer: “We found placement assistance terms, but no employer-side support for the claimed guaranteed job. Obtain written clarification before paying.” Lack of evidence is not a definitive scam label.

Report-to-us stores the candidate’s reviewed allegation privately. Authority reporting prepares a summary and opens the official route; users complete OTP/CAPTCHA. Never display “Reported to police” without an actual acknowledgement. I4C reporting

Use pasted Gmail/WhatsApp/Telegram content initially. A website cannot read those accounts automatically. Voice is not part of the decision loop; optional Polly reads the same explanation with a text fallback.

10. Scale safely and spend efficiently

Traffic and workers: Amplify handles static-site delivery. API functions remain short. SQS absorbs a burst and the worker processes it at a controlled rate. Initially set queue batch size to one and worker concurrency to two, subject to the account quota. Reserve separate capacity for intake/read functions so investigations cannot starve the result page. Increase limits only after measuring Bedrock quotas, latency and cost. AWS SQS/Lambda scaling

What scaling means: if 100 requests arrive, the system can queue them rather than launch 100 expensive investigations. That does not mean immediate answers. Display queue status, cap backlog and reject excess work with retry guidance. The slowest model or external source sets achievable throughput.

Database: use DynamoDB on-demand and key records by random case ID, with separate approved knowledge records. Query keys; do not scan every report. Share cached public policies across users, never private case text. Database scaling still has quotas and hot-key considerations. DynamoDB capacity guidance

No repeated work: content hashes detect duplicate official pages; case revision and request IDs identify duplicate submissions. Conditional writes and an expiring worker lease prevent two deliveries from finalising the same revision. External calls should also tolerate retry; a crash may still repeat a metered model call. SQS is at-least-once, not exactly-once. AWS delivery behaviour

Retries: use bounded exponential backoff for temporary source/model failures. Do not retry invalid input. Set queue visibility from the chosen worker timeout using AWS guidance; send repeatedly failing messages to a dead-letter queue. Save readable failure status when possible, and have status reads expose overdue jobs rather than spin forever. AWS queue configuration

Intake consistency: a database write and queue send are not one transaction. Record enqueue status; if sending fails, return a retrievable error and reuse the same request ID. A later recovery pass can re-enqueue stranded records safely. Do not pretend every accepted database write automatically reached a worker.

Efficient AI: parse domains, validate fields and compare hashes in code. Retrieve only relevant source excerpts. Bound model turns, tokens, input size, tool calls and runtime. Preserve partial evidence when the budget expires. Background refresh has lower priority and its own small concurrency limit.

Privacy and abuse: public samples use fictional data. Real cases need per-case access control even without accounts: use a short-lived random access token, store its hash, and require it on read/update/report calls; do not put it in URLs or logs. Keep it in session storage, not the public repo. This guest design is temporary and offers no account recovery. Apply session/global quotas and input caps; guest tokens alone are not an abuse defence. Add Cognito if persistent accounts become necessary.

Costs: confirm event credits and service/model eligibility first. No paid third-party API is required, but AWS is metered. Add a daily application limit and an administrator kill switch; cloud budget alerts alone do not stop spending. Use short log retention and explicit case expiry. DynamoDB TTL deletion is asynchronous, so expired cases must become unreadable immediately in application code. S3 retention follows the same policy when files are added.

Observability: measure end-to-end latency, model tokens, tool failures, incomplete cases, queue age, retries and actual daily spend. Report what was measured; do not claim production-scale performance from a demo.

11. Work split and integration discipline

Owner	Thursday: first loop	Friday: strengthen	Saturday: finish
A — interface	Input/result UI connected to real API; readable progress and failure	Evidence cards, report review, placement-claim UI	Peer usability checks, accessibility, recording rehearsal
B — agent/tools	Strands worker, official policy retrieval, extraction/result schema	Vacancy adapter, placement checks, held-out benign/scam cases	Controlled knowledge update, evaluation and source audit
C — AWS/release	SAM backend, queue/table, Amplify, model access, CI	Private report storage, token isolation, retries and cost limits	Refresh schedule if ready, deployment checks, release tag

Agree API schemas before splitting. Integrate at hours two, four and six, then at least twice daily. Each person tests another person's component through the deployed app. Everyone owns the final loop, not just their folder.

Sunday: freeze features. Fix only release-blocking defects, record, verify public links and submit. If a late optional feature is unreliable, hide/remove it from the release and describe it as future work.

12. Repo, demo and judging

Use one public repo with web/, backend/, infra/, evals/ and docs/. Issues describe user-visible acceptance criteria. Short feature branches and small PRs let a teammate review actual behaviour. Include screenshots where useful, validation and limitations. Merge passing changes daily; keep honest authorship and history. There is no evidence that judges prefer artificially busy PR activity.

Deploy from main using a restricted AWS role and GitHub OIDC. CI builds the frontend and tests important parsing, case isolation, retries and evaluation regressions. Keep real evidence and secrets out of commits. Tag the submitted version and show its identifier in the app.

README first screen: problem, user, public URL, three-minute video and sample instructions. Then explain architecture, reproducible setup, source freshness, limitations, actual evaluation results, cost observations and what each teammate learned.

Follow the supplied guide's video allocation:

- 0:00–0:20 — user/problem: candidate is about to pay for a promised job.
- 0:20–1:50 — 90 seconds of working product: paste the offer, show actual checks and source evidence, resolve one missing fact if needed, prepare the report. One case is enough.
- 1:50–2:20 — 30 seconds of architecture: show the diagram; explain queue, worker, model, database and one cost decision.
- 2:20–2:45 — failure case: unavailable source or unsupported affiliation produces a useful uncertain result, not an accusation or endless spinner.
- 2:45–3:00 — learning and next step: one concrete lesson and the next improvement. If the knowledge update is built, show its version briefly; otherwise identify it as future work.

Record the screen and real output. Label synthetic evidence, cached sources and any time cuts. No fake integrations or invented accuracy. Test the deployed URL and video in an incognito browser, check repo visibility and leave the deployment running through the review period. Event rubric

13. Release gate

Ship when an unfamiliar teammate can submit an offer, understand the sourced result, recognise uncertainty and prepare a report through the public URL. Verify one scam, one legitimate offer, one unsupported placement claim and one failed-source case. Ensure users cannot read each other's cases and that request limits work.

Measure the remaining quality on a small held-out set: missed risky requests, false accusations, unsupported claims, evidence quality, latency and cost. Use peers to check whether the recommendation changes understanding. Do not call a recommendation "money saved" without evidence.

The earlier research report remains background reading. The delivery priority is now unambiguous: **first public working loop → better evidence and reporting → placement claims → controlled updates → optional convenience features → freeze and demonstrate.**

Appendix: earlier problem shortlist

These directions were explored before narrowing to the recruitment-scam prevention loop. They are retained here so the complete working set remains in one document.

Hackathon problem shortlist

Saved on 16 September 2026. These are candidate problem directions, not selected solutions.

Saved directions

- A small business cannot tell which new market it can enter profitably. Understand what prevents a local business from expanding: demand uncertainty, costs, product adaptation, distribution, compliance, or customer trust. Establish the actual obstacle through real examples before choosing a solution.
- A local seller's expertise does not translate into an effective digital sales experience. Understand how a seller's in-person questions, demonstrations, recommendations, and negotiations help customers buy, and what gets lost when the interaction moves online.

Current selection criteria

- The hackathon has an open theme. Do not constrain exploration to Amazon commerce tracks or previous winners' themes.
- Prioritize a real consumer problem recognizable in Indian family or community life.
- Start with a specific person, situation, obstacle, and consequence, rather than a technology or preselected app concept.
- Seek a meaningful improvement in what people can accomplish, with a focused demonstration feasible for three people in three days.
- AWS deployment, intuitive public access, and a strong demo remain implementation goals after selecting the problem.
- Voice and a dynamically updating interface are possible interaction choices, not requirements that every problem must fit.

Next research step

Gather concrete recent experiences from family, peers, local sellers, and community members. Distinguish observed problems from hypotheses, and inspect existing alternatives before claiming a gap.

Browser-first filter (updated after user feedback)

The final use case must naturally fit a web app hosted on AWS. Exclude directions whose primary value depends on native mobile features, continuous location, push reminders, or frequent on-the-go capture.

Removed from the active exploration list: general caregiving coordination, remote-parent task tracking, accessible outing planning, generic help using phone apps, and camera-led replacement-part identification. Broad relocation and general shopping assistance are also too diffuse without a concrete decision.

Retained for focused research, not yet selected or user-approved: - Families choosing an affordable college/branch and navigating a defined admission decision. - Households deciding whether a rooftop-solar proposal fits their consumption, budget, and requirements. - Households comparing renovation quotations and controlling the cost of scope changes.

Evidence limits: public rules and existing services support the relevance of these domains; unmet demand for each proposed workflow remains to be validated. The two original saved business directions remain on the shortlist.